

Interfacing a Microprocessor to Keyboard

Keyboard আসলে অনেকগুলো switch এর সমষ্টি।

যখন তুমি keyboard-এর কোনো key চাপো:

- একটি switch **ON (Closed)** হয়
- একটি electrical signal তৈরি হয়
- এই signal **microprocessor**-এ যায়
- Microprocessor বুঝতে পারে **কোন key press হয়েছে**

উদাহরণ:

তুমি **A** চাপলে:

- **A** key-এর switch active হবে
- signal CPU/Microprocessor এ যাবে
- Microprocessor বুঝবে **A** press হয়েছে

Keyboard Switch এর Types

4 ধরনের key switch আছে:

1. Mechanical Key Switch
2. Membrane Key Switch
3. Capacitive Key Switch
4. Hall Effect Key Switch

1. Mechanical Key Switch

এটা সবচেয়ে basic type switch।

কিভাবে কাজ করে?

দুইটা metal contact থাকে।

Key press করলে:

- দুইটা metal touch করে
- Circuit complete হয়
- Signal তৈরি হয়

Diagram চিত্রা করো:

Before Press:

Open Circuit
(কোন connection নাই)

After Press:

Closed Circuit
(Connection হয়েছে)

Features

- Phosphor bronze metal ব্যবহার হয়
- Gold plating থাকে (ভালো conduction এর জন্য)
- Spring থাকে key আগের অবস্থায় ফেরানোর জন্য

- In mechanical-switch keys, two pieces of metal are pushed together when you press the key.
- The actual switch elements are often made of a phosphor-bronze alloy with gold plating on the contact areas.
- The key switch usually contains a spring to return the key to the non-pressed position and perhaps a small piece of foam to help damp out bouncing.
- Some mechanical key switches now consist of a molded silicon dome with a small piece of conductive rubber on the underside.
- When a key is pressed, the rubber foam shorts two traces on the printed-circuit board to produce the key pressed signal.

Contact Bounce সমস্যা

এটা important।

Key press করলে অনেক সময় contact একবারে stable হয় না।

এমন হয়:

ON → OFF → ON → OFF → ON

এটাকে **Contact Bounce** বলে।

Microprocessor ভুলে ভাবতে পারে অনেকবার key চাপা হয়েছে।

এজন্য **Debouncing Circuit** লাগে।

Problems

Disadvantages:

- Contact bounce
 - Dust/oxidation হতে পারে
 - পুরনো হলে connection খারাপ হয়
-

Lifetime

- High quality: 1 million keystrokes
 - Silicone dome: 25 million keystrokes
-

2. Membrane Key Switch

এটা Mechanical switch-এর special form।

Structure

3 layer sandwich এর মতো:

- Top layer
- Middle layer (holes)

- Bottom layer
-

কিভাবে কাজ করে?

Key চাপলে:

- Top conductive line
- নিচের conductive line touch করে

Circuit complete হয়। Signal যায়।

~~Membrane key switches~~

- These switches are really a special type of mechanical switches. They consist of a three-layer plastic or rubber sandwich.
- The top layer has a conductive line of silver ink running under each key position.
- The bottom layer has a conductive line of silver ink running under each column of keys.
- When u press a key, you push the top ink line through the hole to contact the bottom ink line.
- The advantages of membrane keyboards is that they can be made as very thin, sealed units.
- They are often used on cash registers in fast food restaurants. The lifetime of membrane keyboards varies over a wide range.

Advantage

- খুব thin
- Cheap
- Waterproof / sealed করা যায়

এজন্য:

- ATM
- Cash register

- Calculator

এগুলোতে ব্যবহার হয়।

Disadvantage

- Lifetime কম হতে পারে
-

3. Capacitive Key Switch

এখানে physical contact নেই।

এটা Capacitance change detect করে।

Capacitance কী?

Capacitor = charge store করে।

দুইটা metal plate থাকলে capacitance হয়।

Key Press করলে

Movable plate fixed plate-এর কাছে আসে। Distance কমে। Capacitance change হয়।

Sense amplifier detect করে:

“Key pressed”

Advantage

- No contact bounce
- No oxidation
- Reliable

Disadvantage

- Extra sensing circuit লাগে
-

Lifetime

20 million keystrokes

- A capacitive key switch has two small metal plates on the printed circuit board and another metal plate on the bottom of a piece of foam.
 - When u press the key, the movable plate is pushed closer to fixed plate. This changes the capacitance between the fixed plates.
 - Sense amplifier circuitry detects this change in capacitance and produce a logic level signal that indicates a key has been pressed.
 - The big advantages of a capacitive switch is that it has no mechanical contacts to become oxidized or dirty.
 - A small disadvantage is the specified circuitry needed to detect the change in capacitance.
 - Capacitive key switches typically have a rated lifetime of about 20 million keystrokes.
-

4. Hall Effect Key Switch কী?

এটা এমন এক ধরনের keyboard switch যেখানে **mechanical contact (metal touch)** লাগে না।

মানে:

- দুইটা metal contact short হয় না
- contact bounce হয় না
- dirt/oxidation সমস্যা নেই

এটা **magnetic field** ব্যবহার করে key press detect করে।

Hall Effect কী?

এটা Hall Effect Principle এর উপর কাজ করে।

Principle:

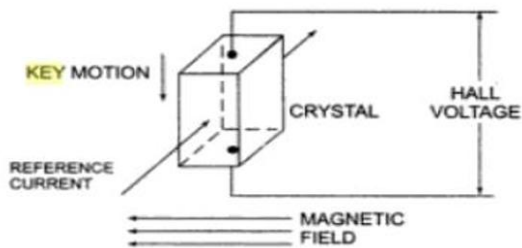
যদি কোনো current-carrying conductor/semiconductor-কে magnetic field এর মধ্যে রাখা হয়, তাহলে conductor-এর দুই পাশে small voltage তৈরি হয়।

এই voltage-কে Hall Voltage বলে।

Components

একটা Hall effect key switch-এ থাকে:

1. Semiconductor crystal
2. Reference current
3. Magnetic field (Magnet)
4. Amplifier circuit



🧠 Step-by-step সহজভাবে

1) আগে (Key চাপা নেই)

- একটা ছোট semiconductor থাকে
- এর ভিতর দিয়ে সাধারণভাবে current চলে
- কিন্তু কোনো output signal তৈরি হয় না

👉 মানে: “কিছু হচ্ছে না”

2) Key চাপলে কী হয়?

- একটি magnet সামনে আসে (বা magnetic field change হয়)
- সেই magnetic field current-এর পাশে লাগে

👉 এখানে একটি গুরুত্বপূর্ণ কথা:

current আর magnetic field একে অপরের সাথে 90° angle এ থাকে

3) Electron গুলোর সাথে কী হয়?

- current মানে electron চলা
- magnetic field ওই electron গুলোকে সোজা পথে যেতে দেয় না
- এক পাশে ঠেলে দেয়

👉 সহজভাবে:

electron গুলো “এক পাশে চাপ খেয়ে সরে যায়”

4) ফলাফল কী?

- এক পাশে বেশি negative charge
- অন্য পাশে positive charge তৈরি হয়

👉 এতে ছোট একটি voltage তৈরি হয়

এটাই হলো:

⚡ **Hall Voltage**

5) তারপর কী হয়?

- এই voltage খুবই ছোট
- তাই amplifier এটাকে বড় করে
- তারপর সেটাকে digital signal বানায়

👉 microcontroller বুঝে ফেলে:
“Key pressed!”

🌀 এক লাইনে সারাংশ

Magnetic field electron কে এক পাশে ঠেলে দেয় → ছোট voltage তৈরি হয় → সেটাকে বড় করে key press detect করা হয়।

কেন Mechanical Contact লাগে না?

কারণ এখানে

✗ Metal touch লাগে না

✓ শুধু magnetic effect দিয়ে key detect হয়

তাই একে **contactless switch** বলে।

Advantage

1. No Contact Bounce

Mechanical switch-এর মতো

ON OFF ON OFF

এমন problem নাই।

2. Very Reliable

Contact dirty হবে না।

Oxidation হবে না।

3. Very Long Life

100 million+ keystrokes

অর্থাৎ

10 কোটি বার press করা যায়।

Disadvantage

Expensive

কারণ লাগে:

- Magnet
- Semiconductor crystal
- Amplifier
- Complex design

তাই cost বেশি।

Real Life Use

Hall effect sensor ব্যবহার হয়:

- Mechanical keyboards
 - Car speed sensors
 - Motor position sensing
 - Industrial machines
 - Motion detection
-

Mechanical vs Hall Effect

Feature	Mechanical	Hall Effect
Contact	Yes	No

Feature	Mechanical	Hall Effect
Bounce	Yes	No
Oxidation	Yes	No
Lifetime	কম	অনেক বেশি
Cost	Cheap	Expensive

Exam Definition

Hall effect key switch is a contactless keyboard switch that uses magnetic field and Hall voltage to detect a key press.

Comparison

Switch	Contact আছে?	Bounce	Cost	Lifetime
Mechanical	Yes	Yes	Low	1M
Membrane	Yes	Some	Low	Varies
Capacitive	No	No	Medium	20M
Hall Effect	No	No	High	100M+

Microprocessor এর সাথে Relation

সব switch-এর main কাজ:

Key press → Electrical Signal → Microprocessor Input

Exam এ Short Definition

Keyboard interfacing হলো এমন একটি process যেখানে keyboard-এর switch press detect করে signal microprocessor-এ পাঠানো হয়, যাতে microprocessor pressed key identify করতে পারে।

~~Keyboard~~ Keyboard Circuit Connections and Interfacing

- In most keyboards, the key switches are connecting in a matrix of rows and columns.
- We will use simple mechanical switches for our examples, but the principle is same for other type of switches.
- Getting meaningful data from a keyboard, it requires the following three major tasks:
 1. Detect a keypress
 2. Debounce the keypress

~~Keyboard~~ Keyboard Circuit Connections and Interfacing

- Three tasks can be done with hardware, software, or a combination of two, depending on the application.

1. Software Keyboard Interfacing:

• *Circuit connection and algorithm :*

- The rows of the matrix are connected to four output port lines.
- The column lines of matrix are connected to four input-port lines.
- To make the program simpler, the row lines are also connected to four input lines.

Keyboard matrix কী?

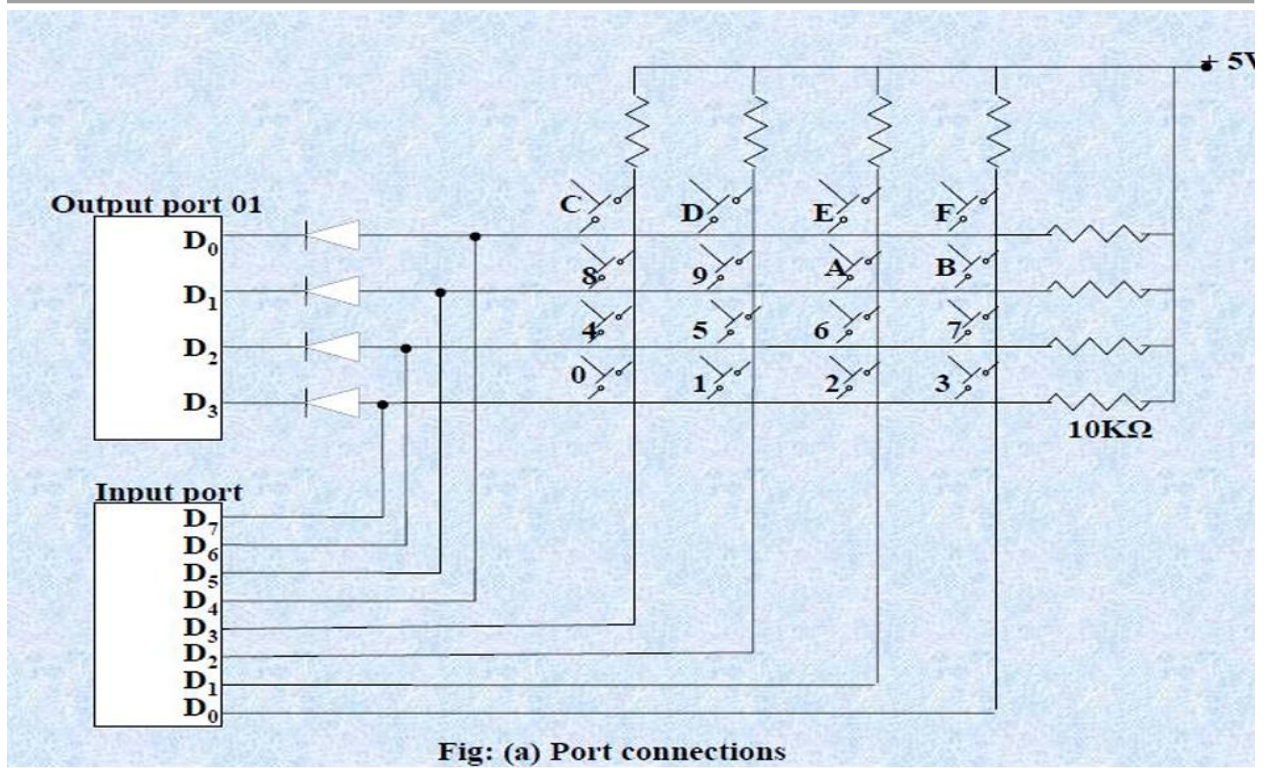
Keyboard এ অনেক key থাকে, কিন্তু প্রতিটা key আলাদা wire দিয়ে না করে:

- Row (সারি)
- Column (কলাম)

এই দুইটা মিলিয়ে একটা grid বানানো হয়।

Basic idea (pull-up resistor কী করছে?)

- Column গুলো সবসময় **HIGH (1)** থাকে +5V pull-up resistor এর কারণে
- মানে কোনো key চাপা না থাকলে সব column = 1



● Step 1: কোনো key চাপা নেই

- Row কিছুই connect করছে না
- Column সব HIGH (1111 ধরো)

👉 Microcontroller দেখে: “No key pressed”

● Step 2: Row গুলো LOW করা হয় (scanning)

Software করে এটা:

- Row1 = 0
- Row2 = 0
- Row3 = 0
- Row4 = 0

👉 মানে সব row LOW করে দেওয়া হলো

🔑 Step 3: Key press হলে কী হয়?

ধরো একটা key চাপা হলো (Row2, Column3)

তাহলে:

- Row2 = LOW
- Key pressed থাকার কারণে Row2 → Column3 connect হয়ে যায়

👉 Column3-এ এখন LOW চলে আসে

🧠 Step 4: MCU কী দেখে?

Microcontroller input port দেখে:

- Column1 = 1
- Column2 = 1
- Column3 = 0 ← এখানে key press detect
- Column4 = 1

👉 Column3 LOW মানে key pressed

🎯 Step 5: Key identify করা

যেহেতু তুমি জানো:

- কোন row LOW ছিল (Row2)
- কোন column LOW এসেছে (Column3)

👉 তাই key ঠিকভাবে identify করা যায়:
(Row2, Column3) = কোন নির্দিষ্ট key



Step 6: Scanning loop (continuous check)

Program বারবার করে:

1. সব row LOW করে
2. column check করে
3. যদি সব column HIGH হয় → কোনো key নাই
4. না হলে → key detect



Two-key lockout কী?

মানে:

- এক সময় একটাই key ঠিকভাবে detect করা হয়
- যদি একসাথে 2টা key চাপা হয়, system confuse হতে পারে



এক লাইনে সারাংশ

Row গুলো LOW করা হয় → column check করা হয় → যে column LOW হয় সেটাই pressed key identify করে।

Two-Key Lockout

Important.

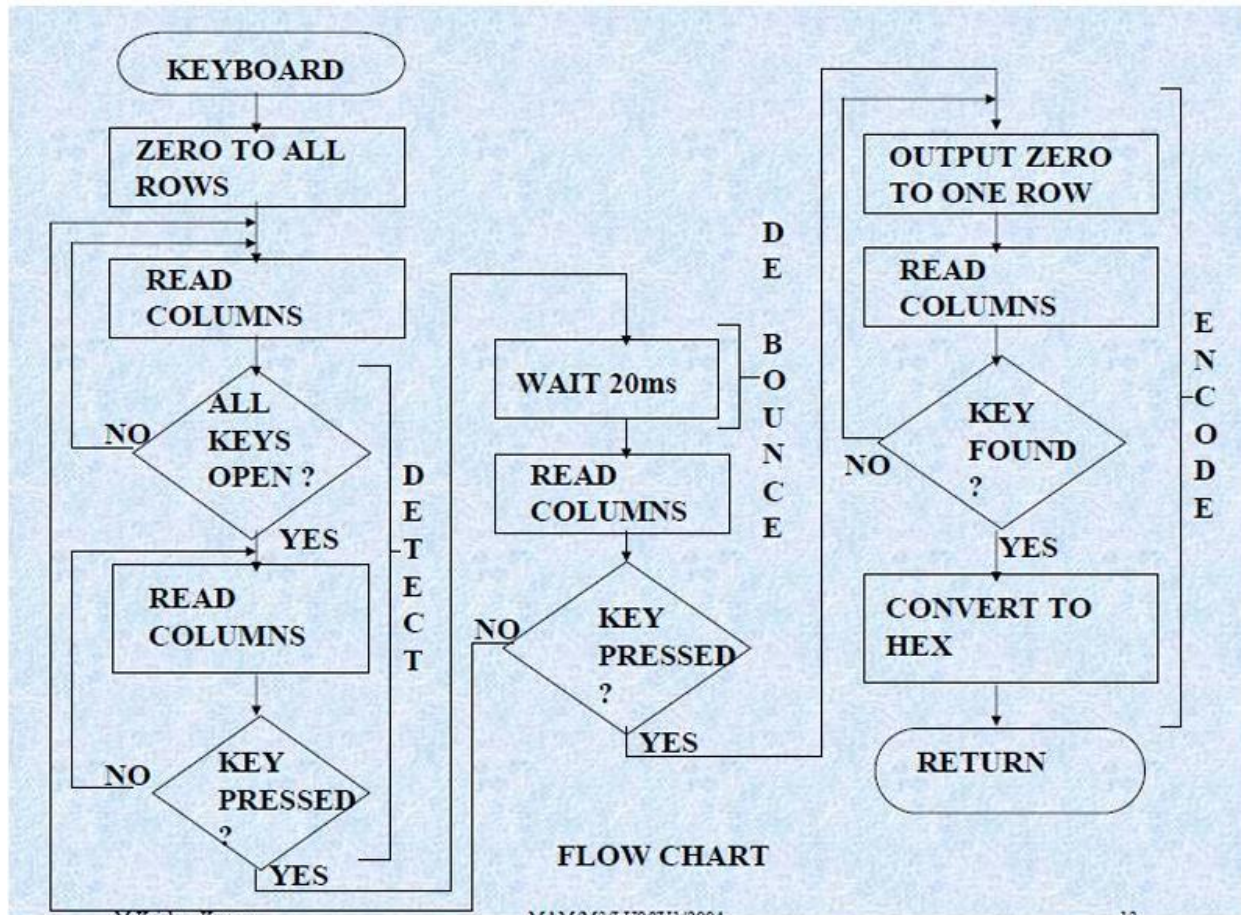
আগের key release না হওয়া পর্যন্ত

নতুন key accept করবে না।

এটাই:

Two-key lockout

একসাথে দুই key press prevent করে।



এটা পুরোটা আসলে keyboard scanning + debounce + key identification algorithm। আমি ধাপে ধাপে খুব সহজভাবে বুঝাচ্ছি।

1) “No key pressed” থেকে “Key press detect” loop

আগে কী হয়?

- Column সবসময় HIGH (1)
- Program বারবার check করে

👉 যতক্ষণ সব column = HIGH
মানে: কোনো key চাপা হয়নি

যখন key চাপা হয়

- কোনো একটা column LOW হয়ে যায়

👉 Program বুঝে:
“Key pressed!”

2) Debounce (20 ms delay) কেন লাগে?

Key চাপলে অনেক সময়:

- একবার চাপলেও signal “কাঁপে” (bounce)
- মানে দ্রুত ON/OFF হয়

👉 তাই program:

- 20 ms wait করে

তারপর আবার check করে

✓ 3) Confirm করা (important step)

20 ms পরে আবার দেখে:

- যদি column আবার সব HIGH হয়
👉 এটা ছিল noise (false signal)
 - যদি এখনও LOW থাকে
👉 এটা real key press
-

4) Row + Column বের করা (actual key identify)

এখন program key খুঁজে বের করে:

কিভাবে?

1. একবারে শুধু 1টা row LOW করা হয়
2. column read করা হয়

Example:

ধরো:

- Row1 LOW → column HIGH → key নেই
- Row2 LOW → column3 LOW → key found!

👉 মানে:

Key = Row2 + Column3

5) Why row by row check?

কারণ:

- একসাথে সব row LOW করলে বোঝা যাবে না key কোথায়
 - তাই একটার পর একটা row test করা হয়
-

6) Hex code / mapping

Keyboard এ প্রতিটা (row, column) এর একটা code থাকে

👉 উদাহরণ:

Row2 + Column3 → 0x2B (ধরা যাক)

Program এই mapping থেকে final key বের করে

7) শেষ অংশ (memory + error trap idea)

তোমার note-এ যেটা আছে:

- Program 0000H থেকে FFFFH পর্যন্ত search করে match খুঁজতে পারে
- যদি match না পায় → error

👉 মানে:

সব possible value check করে correct key খোঁজার চেষ্টা



কেন error trap দরকার?

কারণ:

- noise
- wrong signal
- loose key press

👉 এগুলোর কারণে ভুল detect হতে পারে



পুরোটা এক লাইনে

Program প্রথমে wait করে key press → debounce করে confirm করে → তারপর row-by-row scan করে column detect করে exact key বের করে।

- This code would not match any of the row-column codes in the table, so after all the values in the table were checked, assigned register in program would be decremented from 0000H to FFFFH.
 - The compare decrement cycle would continue through 65,536 memory locations until, by change the value in a memory location matched the row-column code.
 - The contents of the lower byte register at that point would be passed back to the calling routine.
 - The changes are 1 in 256 that would be the correct value for one of the pressed keys.
 - You should keep an error trap in a program whenever there is a chance for it.
-

যদি Match না করে?

যদি code table-এর কোনো valid row-column code-এর সাথে না মেলে,

তাহলে problem।

মানে:

- Noise হতে পারে
- ভুল signal হতে পারে
- Invalid key code হতে পারে

তখন কী হয়?

Program compare করতে করতে register decrement করে:

0000H

থেকে

FFFFH

Decrement মানে?

কমতে থাকে:

0000H

↓

FFFFH

↓

FFFEH

↓

FFFDH

এভাবে।

65,536 memory locations কেন?

কারণ:

16-bit register

$$2^{16} = 65,536$$

তাই মোট 65,536 possible value check করতে পারে।

“match পেয়ে গেলে কী হয়?”

ধরো keyboard থেকে একটা (row, column) code বের হলো।

এখন program কী করে:

- একটা memory table আছে যেখানে সব key এর code রাখা
- program একে একে সেই table scan করে

👉 যদি কোনো জায়গায় exact match পায়:

- সেই value ধরে নেয় “এটাই pressed key”
- ওই value (lower byte) return করে দেয় calling function-এ

📌 মানে:

Match = key successfully identified

2) “1 in 256 chance” মানে কী?

Lower byte মানে 8-bit value:

👉 possible values:

- 00000000 থেকে 11111111
- total = 256 values

এখন যদি randomভাবে match খুঁজে পায়, তাহলে:

👉 256টা জায়গার মধ্যে 1টা correct হতে পারে
তাই probability বলা হচ্ছে:

1/256 chance = accidental correct match

⚠️ 3) এটা risky কেন?

কারণ system যদি:

- noise ধরে ভুল code বানায়
- বা scanning error হয়

👉 তখন এমন একটা value তৈরি হতে পারে যেটা accidentally table-এর সাথে মিলে যায়

❌ ফল:

- তুমি চাপছো “5”
- system ধরে নিল “8”

👉 এটাকে বলে **false positive**

🛡️ 4) Error trap কেন দরকার?

Error trap মানে safety check system।

👉 এটা নিশ্চিত করে:

- ভুল value accept না করা
 - invalid data discard করা
 - system crash না করা
-

🔄 Error trap কীভাবে কাজ করে?

যদি match না নিশ্চিত হয়:

- value reject করবে ✕
 - আবার scan করবে 🔄
 - অথবা ignore করবে ⏩
 - program hang হওয়া আটকাবে 🚫
-

🎯 এক লাইনে সারাংশ

Program যদি table scan করে accidental match পায়, সেটাকেই key ধরে নেয়, কিন্তু noise বা ভুলের কারণে wrong match হওয়ার chance থাকে, তাই error trap দিয়ে system safe রাখা হয়।

2. Keyboard Interfacing with Hardware:

- For the system where the CPU is too busy to be bothered doing these tasks in software, an external device is used to do them.
- One of a MOS device which can be do this is the General Instruments AY5-2376 which can be connected to the rows and columns of a keyboard switch matrix.
- The AY5-2376 independently detects a keypress by cycling a low down through the rows and checking the columns.
- When it finds a key pressed, it waits a debounce time.
- If the key is still pressed after the debounce time, the AY5- 2376 produces the 8-bit code for the pressed key and send it out to microcomputer port on 8 parallel lines.
- The microcomputer knows that a valid ASCII code is on the data lines, the AY5-2376 outputs a strobe pulse.
- The microcomputer can detect this strobe pulse and read in ASCII code on a polled basis or it can detect the strobe pulse on an **interrupt basis**.
- With the interrupt method the microcomputer doesn't have to pay any attention to the keyboard until it receives an interrupt signal.
- So this method uses very little of the microcomputer time.
- The AY5-2376 has a feature called **two-key rollover**.
- This means that if two keys are pressed at nearly the same time, each key will be detected, debounced and converted to ASCII.
- The ASCII code for the first key and a strobe signal for it will be sent out then the ASCII code for the second key and a strobe signal for it will be sent out and compare this with two-key lockout.

Hardware Keyboard Interfacing কী?

আগে যেটা দেখেছি (Software method):

- CPU নিজে Detect করে
- Debounce করে
- Encode করে

👉 এতে CPU অনেক ব্যস্ত হয়ে যায়।

Hardware Method এ কী হয়?

এই কাজগুলো একটা external IC করে দেয়।

এখানে সেই IC হলো:

AY5-2376 (Keyboard Encoder IC)

AY5-2376 কী করে?

এই IC পুরো keyboard handling নিজে করে:

1. Key Detect করে
2. Debounce করে
3. ASCII code তৈরি করে

তারপর CPU-কে শুধু ready data দেয়।

কিভাবে কাজ করে?

Step 1: Row Scanning

IC নিজেই:

- এক এক করে row-তে LOW পাঠায়
- column check করে

👉 একদম software scanning-এর মতো, কিন্তু hardware-এ হচ্ছে।

Step 2: Key Detect

যখন কোনো key press হয়:

- row এবং column connect হয়
 - IC বুঝে key press হয়েছে
-

Step 3: Debounce

IC wait করে কিছু সময় (debounce time)

তারপর check করে:

- key এখনও press আছে?

✓ থাকলে → valid key

✗ না থাকলে → ignore

Step 4: ASCII Code তৈরি

Valid key হলে:

- IC নিজেই 8-bit ASCII code তৈরি করে

যেমন:

- 'A' → 41H
 - 'I' → 31H
-

Step 5: Data Output

এই ASCII code:

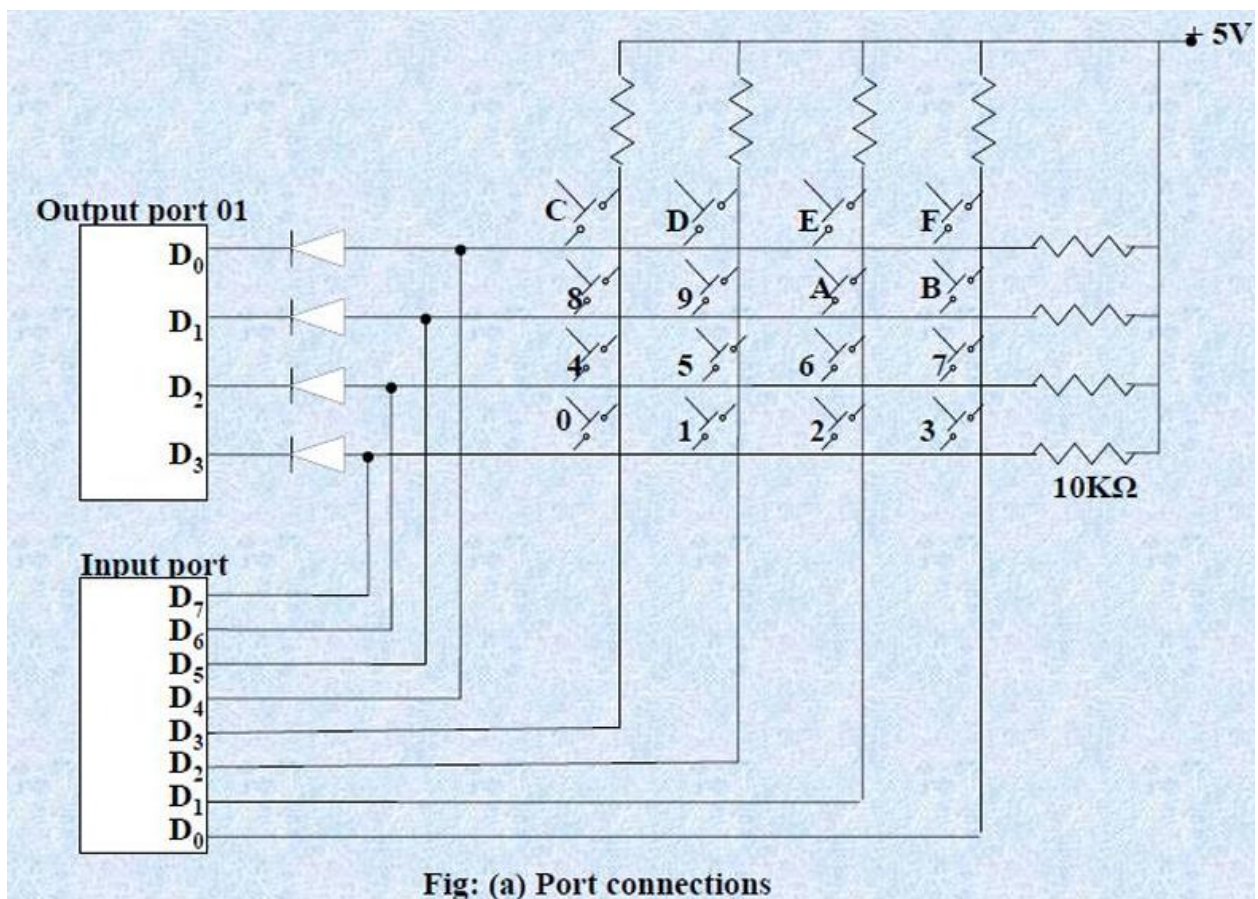
👉 ৪টা parallel line দিয়ে microprocessor-এ পাঠায়

Step 6: Strobe Signal

IC একটা **Strobe Pulse** দেয়।

👉 এটা signal দেয়:

“Data ready, এখন পড়ো”



Microprocessor কীভাবে পড়ে?

২ভাবে পড়তে পারে:

1. Polling Method

CPU বারবার check করে:

“data এসেছে নাকি?”

👉 inefficient (সময় নষ্ট হয়)

2. Interrupt Method (Better)

IC strobe pulse দিলে:

👉 interrupt trigger হয়

CPU তখনই কাজ বন্ধ করে keyboard data পড়ে।

- ✓ Efficient
 - ✓ Fast
 - ✓ CPU free থাকে
-

Advantage

- ✓ CPU-র কাজ কম
 - ✓ Faster response
 - ✓ Reliable
 - ✓ Software সহজ
-

Two-Key Rollover

এটা খুব important feature।

মানে কী?

যদি দুইটা key একসাথে press করো:

👉 দুটোই detect হবে

Example

তুমি চাপলে:

A এবং B একসাথে

তখন:

1. A detect হবে → ASCII + strobe
 2. B detect হবে → ASCII + strobe
-

Compare with Two-Key Lockout

Feature	Meaning
Two-key lockout	একটার পর আরেকটা key না
Two-key rollover	দুইটাই detect হয়

Difference সহজভাবে

- Lockout → একটাই key allowed
- Rollover → multiple key allowed

~~Example~~

- Interface a 4 * 4 keyboard with 8086 using 8255 and write an ALP for detecting a key closure and return the key code in AL. The debounce period for a key is 10ms. Use software debouncing technique. DEBOUNCE is an available 10ms delay routine.

- একটি 4×4 কিবোর্ডকে 8255-এর মাধ্যমে 8086 প্রসেসরের সাথে কানেক্ট করতে হবে।
- একটি অ্যাসেম্বলি ল্যাঙ্গুয়েজ প্রোগ্রাম (ALP) লিখতে হবে যা কিবোর্ডের কোনো কী (Key) প্রেস করা হয়েছে কি না তা ডিটেক্ট করবে এবং প্রেস করা কী-এর ইউনিক Key Code-টি AL রেজিস্টারে রিটার্ন করবে।
- মেকানিক্যাল সুইচের বাউন্সিং সমস্যা এড়াতে Software Debouncing টেকনিক ব্যবহার করতে হবে (যার জন্য ১০ মিলি-সেকেন্ডের একটি রেডিমেড DEBOUNCE ডিলে রুটিন দেওয়া আছে)।

- **Solution:** Port A is used as output port for selecting a row of keys while Port B is used as an input port for sensing a closed key.
- Thus the keyboard lines are selected one by one through port A and the port B lines are polled continuously till a key closure is sensed.
- The routine DEBOUNCE is called for key debouncing.
- The key code is depending upon the selected row and a low sensed column.

২. ইন্টারফেসিং লজিক ও মেকানিজম (image_443f82.png)

কিবোর্ডের বাটনগুলো ম্যাট্রিক্স আকারে (Row এবং Column) সাজানো থাকে। দ্বিতীয় ইমেজ (image_443f82.png) অনুযায়ী এটি যেভাবে কাজ করে:

- **Port A (Output Port):** কিবোর্ডের রোগুলোকে (Rows) সিলেক্ট করার জন্য ব্যবহার করা হয়।
- **Port B (Input Port):** কোন কলামের কী প্রেস করা হয়েছে (Sense) তা রিড করার জন্য ব্যবহার করা হয়।
- **কাজ করার পদ্ধতি:** প্রসেসর এক এক করে প্রতিটি Row-কে গ্রাউন্ড বা লো (0) করবে এবং কলামগুলোর স্টেট চেক (Polling) করবে। কোনো বাটন প্রেস করা হলে সংশ্লিষ্ট কলামে একটি 0 বা Low সিগন্যাল পাওয়া যাবে।

৩. সার্কিট ডায়াগ্রাম ও কনফিগারেশন

তৃতীয় ইমেজ থেকে আমরা হার্ডওয়্যার এবং অ্যাড্রেসিং-এর ডিটেইলস পাই:

ক. সার্কিট ও পোর্ট কানেকশন:

ডায়োগ্রামে দেখুন, Rows ($PA_0 - PA_3$) এবং Columns ($PB_0 - PB_3$) যুক্ত আছে। বাকি হায়ার অর্ডার লাইনগুলো ($PA_4 - PA_7$ এবং $PB_4 - PB_7$) অব্যবহৃত রাখা হয়েছে।

- কলাম লাইনের সাথে $+5V$ এবং $10k\Omega$ এর পুল-আপ (Pull-up) রেজিস্টর লাগানো আছে। এর মানে হলো, স্বাভাবিক অবস্থায় (কোনো কী প্রেস না করলে) সব কলাম লাইন হাই বা 1 থাকবে।
- যখন কোনো Row-তে 0 পাঠানো হবে এবং সেই Row-এর কোনো কী প্রেস করা হবে, তখন কলাম লাইনটি সরাসরি গ্রাউন্ডের সাথে শর্ট হয়ে যাবে এবং রিড করার সময় সেখানে 0 পাওয়া যাবে।

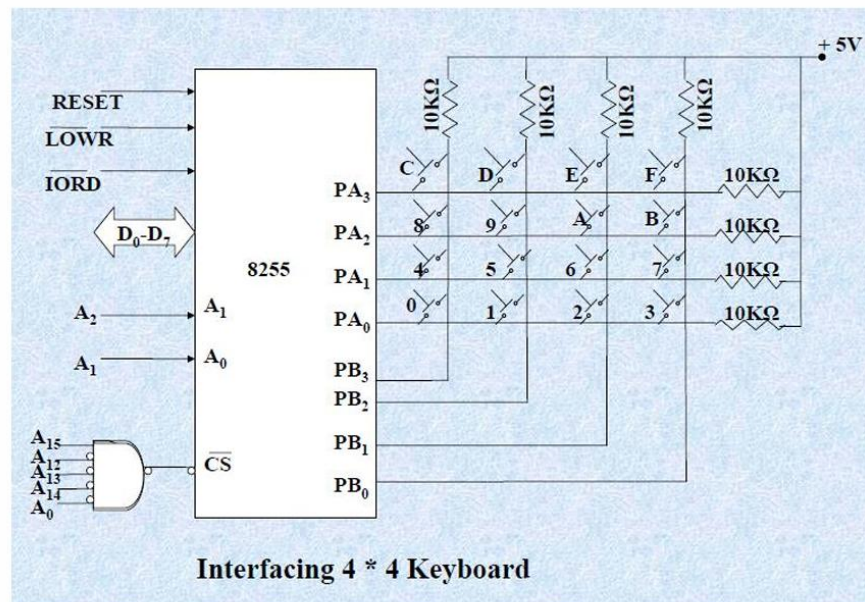
খ. পোর্ট অ্যাড্রেস এবং কন্ট্রোল ওয়ার্ড:

- **Port A Address:** 8000H
- **Port B Address:** 8002H
- **CWR (Control Word Register) Address:** 8006H
- **Control Word:** 82H (বাইনারিতে 10000010)। এটি 8255-কে নির্দেশ দেয় যেন Port A আউটপুট হিসেবে এবং Port B ইনপুট হিসেবে কাজ করে।

গ. কী ডিবিন্ড বা Key Debounce:

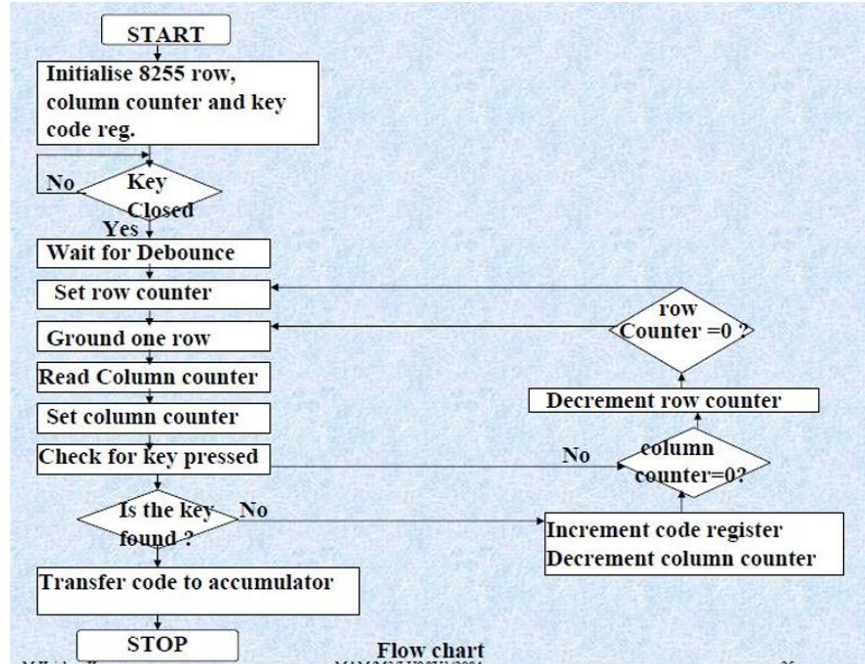
- মেকানিক্যাল বাটন প্রেস করলে মেটাল কন্ট্যাক্টগুলো সাথে সাথে স্থির হয় না, বরং কয়েক মিলি-সেকেন্ডের জন্য অন-অফ (Bounce) হতে থাকে। একে ফলস ট্রিগার না ভেবে সঠিক রিডিং নেওয়ার জন্য বাটন ডিটেক্ট হওয়ার পর ১০ মিলি-সেকেন্ড অপেক্ষা করে (DEBOUNCE রুটিন কল করে) পুনরায় কনফার্ম করা হয়।

Keyboard Interfacing with Hardware



- The higher order lines of port A and port B are left unused.
- The address of port A and port B will respectively 8000H and 8002H while address of CWR will be 8006H.
- The control word for this problem will be 82H.
- Code segment CS is used for storing the program code.
- **Key Debounce** : Whenever a mechanical push-button is pressed or released once, the mechanical components of the key do not change the position smoothly, rather it generates a transient response .

Flow Chart



আসুন সহজ বাংলায় প্রতিটা ধাপ ধাপে ধাপে বুঝে নিই:

১. প্রিপারেশন এবং ইনিশিয়ালাইজেশন (Start & Initialize)

- **START:** প্রোগ্রাম এখান থেকে শুরু হয়।
- **Initialise 8255 row, column counter and key code reg.:** শুরুতে 8255 PPI-কে কনফিগার করা হয় (Port A আউটপুট এবং Port B ইনপুট হিসেবে)। একই সাথে প্রসেসরের ভেতরে ৩টি জিনিস সেট করা হয়:
 - **Row Counter:** মোট ৪টি রো ট্র্যাক করার জন্য (ভ্যালু = ৪)।
 - **Column Counter:** প্রতি রো-তে ৪টি কলাম ট্র্যাক করার জন্য (ভ্যালু = ৪)।
 - **Key Code Register:** এটি শুরুতে 00H দিয়ে সেট করা হয়। বাটন খোঁজার সময় এটি এক এক করে বাড়তে থাকবে, যা শেষ পর্যন্ত প্রেস করা বাটনটির আসল মান (যেমন: 0 থেকে F) নির্দেশ করবে।

২. কী প্রেস ডিটেকশন ও ডিবিন্ডিং (Key Closed? & Debounce)

- **Key Closed?:** প্রোগ্রামটি লুপ আকারে চেক করতে থাকে কিবোর্ডের কোনো একটি বাটন প্রেস করা হয়েছে কি না। যতক্ষণ কোনো বাটন প্রেস না হবে, এটি "No" লাইনে ঘুরতে থাকবে (Polling)। যখনই কোনো বাটন প্রেস হবে, এটি "Yes" কন্ডিশনে চলে যাবে।
- **Wait for Debounce:** বাটন প্রেস হওয়া মাত্রই ১০ মিলি-সেকেন্ডের ডিবিন্ডিং ডিলে দেওয়া হয়, যাতে সুইচের মেকানিক্যাল ভাইব্রেশনের কারণে কোনো ভুল রিডিং না আসে।

৩. রো এবং কলাম স্ক্যানিং লুপ (Row & Column Scan)

- **Set row counter & Ground one row:** ডিবিন্ডিং শেষে Row Counter সেট করা হয় এবং প্রথম রো-টিকে 0 (Ground) করা হয় (বাকি ৩টি রো 1 থাকে)।
- **Read Column counter & Set column counter:** প্রথম রো গ্রাউন্ড করার পর কলাম রিড করা হয় এবং কলাম কাউন্টার সেট করা হয়।
- **Check for key pressed & Is the key found?:** এবার চেক করা হয় এই নির্দিষ্ট রো-এর কোনো কলামে 0 বা লো সিগন্যাল পাওয়া যাচ্ছে কি না।

৪. লজিক ডিসিশন (বাটন খুঁজে পাওয়া অথবা না পাওয়া)

ক) যদি বাটন খুঁজে না পাওয়া যায় ("No" পথ):

যদি প্রথম রো-এর কোনো কলামেই বাটন প্রেস করা না থাকে, তবে প্রোগ্রাম নিচের ডানদিকের লুপে চলে যায়:

1. **Increment code register & Decrement column counter:** কী কোড রেজিস্টারের মান ১ বাড়ানো হয় এবং কলাম কাউন্টার ১ কমানো হয়।
2. **column counter = 0?:** চেক করা হয় এই রো-এর সব কলাম চেক করা শেষ কি না। শেষ না হলে (No) আবার পরের কলাম চেক করে।
3. **Decrement row counter & row Counter = 0?:** যদি একটি রো-এর সব কলাম চেক করা শেষ হয়ে যায় (Yes), তখন রো কাউন্টার ১ কমানো হয়। এরপর চেক করা হয় সব রো দেখা শেষ কি না। শেষ না হলে (No), পরবর্তী রো-কে গ্রাউন্ড (0) করে আবার কলামগুলো স্ক্যান করা শুরু হয়।

সহজ কথায়: বাটন না পাওয়া পর্যন্ত প্রোগ্রামটি রো 0, রো 1, রো 2, রো 3 এভাবে এক এক করে স্ক্যান করতে থাকে এবং প্রতিবার বাটন না পেয়ে সামনে আগানোর সময় Key Code Register-এর মান ১ করে বাড়িয়ে দেয় (যেমন: 0, 1, 2, 3... এভাবে)।

খ) যদি বাটন খুঁজে পাওয়া যায় ("Yes" পথ):

- স্ক্যান করতে করতে ঠিক যে মুহূর্তে কান্ডাকৃত বাটনটি (যেখানে 0 পাওয়া গেছে) মিলে যাবে, তখন **Is the key found?** কন্ডিশনটি "Yes" হবে।
- **Transfer code to accumulator:** বাটনটি মিলে যাওয়ার সাথে সাথে লুপ বন্ধ হয়ে যাবে। ঐ মুহূর্তে Key Code Register-এ যে মানটি জমা হয়ে আছে (যা নিখুঁতভাবে প্রেস করা বাটনের পজিশন নির্দেশ করে), সেটি প্রসেসরের Accumulator (AL রেজিস্টার)-এ ট্রান্সফার বা কপি করে দেওয়া হয়।
- **STOP:** সঠিক কী কোডটি পাওয়ার পর প্রোগ্রামটি সফলভাবে শেষ হয়।

Key Debounce

যখন একটি mechanical push-button একবার press বা release করা হয়, তখন এর mechanical parts smooth ভাবে position change করে না।

বরং এটি একটি **transient response** (অস্থায়ী ওঠানামা) তৈরি করে।

এই transient variation-গুলো microprocessor অনেক সময় **multiple key press** হিসেবে ভুল বুঝতে পারে এবং সেই অনুযায়ী response দিতে পারে।

Debouncing মানে কী?

Bounce থামিয়ে আসল valid key press detect করা।

1. Hardware Debouncing

এখানে external hardware ব্যবহার করা হয়।

যেমন:

- Bistable Multivibrator (Flip-Flop)

Switch bounce করলেও output clean একটাই signal দেয়।

আগে (bounce):

101010

Hardware debounce এর পর:

000000

একটাই proper signal।

Advantage

- Fast
- Reliable

Disadvantage

- Extra hardware লাগে
-

2. Software Debouncing

এখানে hardware না, program দিয়ে solve করা হয়।

ধাপগুলো:

Step 1

Key press detect হলো।

Logic 0 detected

Step 2

সাথে সাথে accept করবে না।

10 ms wait করবে:

CALL DEBOUNCE

Step 3

আবার check করবে।

যদি এখনও pressed থাকে,

তাহলে valid key।

যদি না থাকে,

bounce ছিল।

Example

ধরো key চাপলে 5ms bounce করে।

0 1 0 1 0

তুমি 10ms wait করলে bounce শেষ।

তারপর check:

0

এখন stable।

Valid key press.

Key Press হলে Logic কী হবে?

- Key press হলে microprocessor **logic 0 (low)** read করবে।

High Precision Applications এ দুইটি Option

Option 1

- একাধিক **8-bit port** ব্যবহার করা।
- এক এক করে port read/write করা।
- তারপর multibyte data তৈরি করা।

Option 2

- দুইটি 8-bit port মিলিয়ে **16-bit port** বানানো।
 - তারপর 16-bit read/write operation ব্যবহার করা।
-